

MINISTRY OF SCIENCE AND HIGHER EDUCATION OF THE RUSSIAN FEDERATION  
FEDERAL STATE AUTONOMOUS EDUCATIONAL INSTITUTION OF HIGHER  
EDUCATION  
"NOVOSIBIRSK NATIONAL RESEARCH UNIVERSITY  
STATE UNIVERSITY"  
(NOVOSIBIRSK STATE UNIVERSITY, NSU)

09.03.03 - Applied Computer Science

Focus (profile): Applied Artificial Intelligence

**TERM PAPER**

**Authors**

Job topic:

**"SNAKE GAME"**

Баландин Константин 25946

Адуев Егор 25946

Шевченко Андрей 25946

Novosibirsk, 2026

## Содержимое

|                                                                      |    |
|----------------------------------------------------------------------|----|
| 1. Термины и сокращения .....                                        | 3  |
| 2. Введение .....                                                    | 4  |
| 3. Назначение и область применения.....                              | 4  |
| 4. Технические характеристики.....                                   | 4  |
| 5. Описание схем.....                                                | 6  |
| 5.1. Главная схема (main) .....                                      | 6  |
| 5.2. Игровой модуль (Game) .....                                     | 7  |
| 5.3. Модуль движения (Movement) .....                                | 10 |
| 5.4. Модули управления клавиатурой (Keyboard, EditKeyboardKeys)..... | 15 |
| 5.5. Схема генерации яблок (GenerateApple).....                      | 18 |
| 5.6. Схема проверки поедания яблока (CheckEatApple).....             | 19 |
| 5.7. Счетчик очков (CounterScore).....                               | 19 |
| 5.8. Схемы отображения (DataScreen, UpdateScreen) .....              | 20 |
| 5.9. Схемы проверки столкновений .....                               | 22 |
| 5.10. Модули игровых состояний .....                                 | 24 |
| 5.11. Процессорный модуль (CDM-8).....                               | 25 |
| 6. Программная часть .....                                           | 27 |
| 6.1. Структура программы “snake.asm” .....                           | 27 |
| 6.2. Взаимодействие программы со схемами .....                       | 28 |
| 7. Инструкция по запуску.....                                        | 34 |
| 8. Заключение .....                                                  | 35 |
| 9. Источники, использованные при разработке .....                    | 36 |

# 1. Термины и сокращения

|                   |                                                                               |
|-------------------|-------------------------------------------------------------------------------|
| CDM-8 Mark 4      | Coco-de-Mer 8 – 8битный процессор, версия Mark 4                              |
| Logisim           | Программа для моделирования цифровых логических схем                          |
| Игровое поле      | Область отображения объектов игры (змейка, яблоко, границы)                   |
| I/O               | Input/Output – ввод/вывод                                                     |
| Memory-mapped I/O | Ввод-вывод через адресное пространство памяти                                 |
| Схема             | Отдельная логическая схема в Logisim, выполняющая специализированную функцию  |
| RAM               | Random Access Memory - оперативная память                                     |
| ASCII             | Стандарт кодирования символов клавиатуры                                      |
| MUX               | Мультиплексор - элемент, выбирающий один из нескольких входов                 |
| DMX               | Демультимплексор - элемент, направляющий сигнал на один из нескольких выходов |
| XOR               | Логический элемент "исключающее ИЛИ"                                          |
| Координаты        | Набор значений (X, Y), определяющих положение игровых объектов на поле        |

## 2. Введение

Данная пояснительная записка предназначена для описания технической реализации проекта «Snake», разработанного в среде Logisim с использованием цифровой логики и схмотехнического подхода. Проект представляет собой аппаратно-логическую реализацию классической игры «Змейка», в которой игрок управляет движением объекта на игровом поле, собирает яблоки и увеличивает игровой счёт.

Основной особенностью проекта является реализация игровой логики с использованием отдельных взаимосвязанных схем, отвечающих за движение, отображение, обработку ввода, генерацию объектов и управление состояниями игры. Управление последовательностью игровых действий осуществляется программой на ассемблере CDM-8, которая выполняется на 8-битном процессоре, встроенном в схему. Взаимодействие процессора с аппаратными схемами происходит через порты ввода-вывода (memory-mapped I/O).

## 3. Назначение и область применения

Проект «Snake» предназначен для демонстрации возможностей реализации игровой логики средствами цифровой схмотехники. Игра позволяет управлять змейкой, перемещать её по игровому полю, собирать яблоки и отслеживать количество набранных очков.

Проект может использоваться в образовательных целях при изучении цифровой логики, принципов построения конечных автоматов, хранения координатных данных и взаимодействия логических схем.

## 4. Технические характеристики

Проект реализован с использованием среды Logisim для проектирования цифровой логики, а также Assembler CDM-8 для программного управления отдельными игровыми процессами. Программная логика игры записана в виде набора инструкций на языке ассемблера и выполняется процессором CDM-8.

В проекте используется процессорная архитектура, при которой часть логики реализуется аппаратными схемами Logisim, а часть - программным способом через инструкции CDM-8. Ассемблерный код отвечает за обработку игрового цикла, управление движением змейки, генерацию яблок и взаимодействие с логическими схемами.

Основные параметры:

Процессор: CDM-8 Mark 4, 8-битный

Разрядность координат: 4 бита

Счётчик очков: 3 разряда (максимум 999)

Управление: клавиши W/A/S/D

Игровые экраны: 3 (старт, игра, конец)

Проект реализован в среде Logisim и состоит из набора взаимосвязанных схем, каждая из которых отвечает за отдельный аспект функционирования игры. В основе архитектуры лежит модульный принцип проектирования.

Основные функции системы:

обработка пользовательского ввода;

управление движением змейки;

хранение и обновление координат;

генерация яблок;

проверка столкновений;

подсчёт очков;

вывод изображения на экран;

переключение игровых состояний.

Логика игры разделена на независимые подсистемы, что упрощает отладку и модернизацию проекта.

## 5. Описание схем

### 5.1. Главная схема (main)

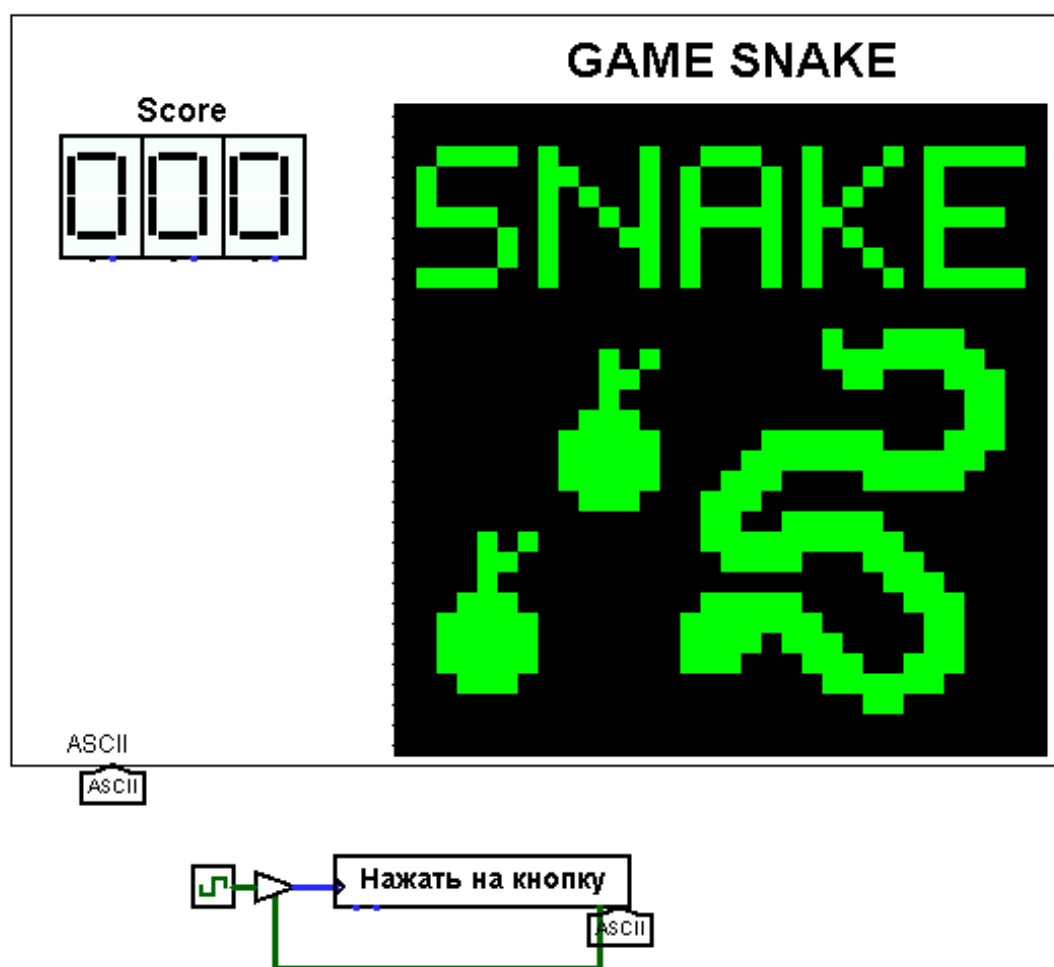


Рисунок 1. Главная схема

Главная схема (main) представляет собой верхний уровень проекта и объединяет все функциональные модули игры. Данный уровень включает в себя следующие элементы пользовательского интерфейса :

Игровое поле - светодиодная матрица, на которой отображается змейка, яблоко и границы поля.

Счётчик очков - трёхразрядный дисплей, отображающий текущее количество съеденных яблок.

Клавиатура (ASCII) - компонент ввода, принимающий нажатия клавиш (w,a,s,d) для управления направлением движения змейки.

Кнопка активации клавиатуры - перед началом игры необходимо нажать кнопку под дисплеем для активации ввода с клавиатуры.

## 5.2. Игровой модуль (Game)

Модуль Game реализует основную игровую логику. В нём осуществляется управление игровым циклом, обработка действий игрока и взаимодействие с остальными модулями.

Модуль получает на вход ASCII-код нажатой клавиши с клавиатуры и тактовый сигнал. На выходе формирует изображение игрового поля и текущий счёт.

Внутри модуля расположены:

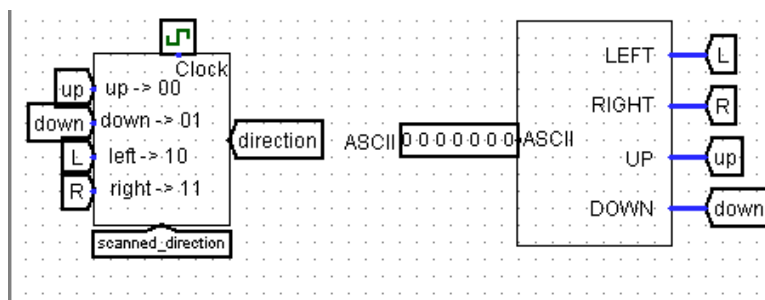


Рисунок 2. Блок обработки ввода.

Блок обработки ввода - преобразует нажатую клавишу (w,a,s,d) в 2-битный код направления

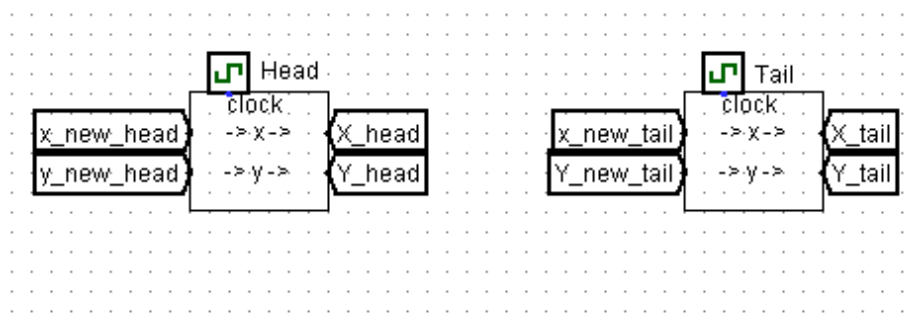


Рисунок 3. Модуль SaveCoordinates (сохранение координат)

Регистры координат головы и хвоста змейки.

Регистры Head и Tail хранят текущие координаты головы и хвоста змейки. По тактовому сигналу (clock) принимают новые координаты от модуля Movement и фиксируют их.

Входные значения (X\_head, Y\_head, X\_tail, Y\_tail) используются как входные данные для следующего игрового цикла.

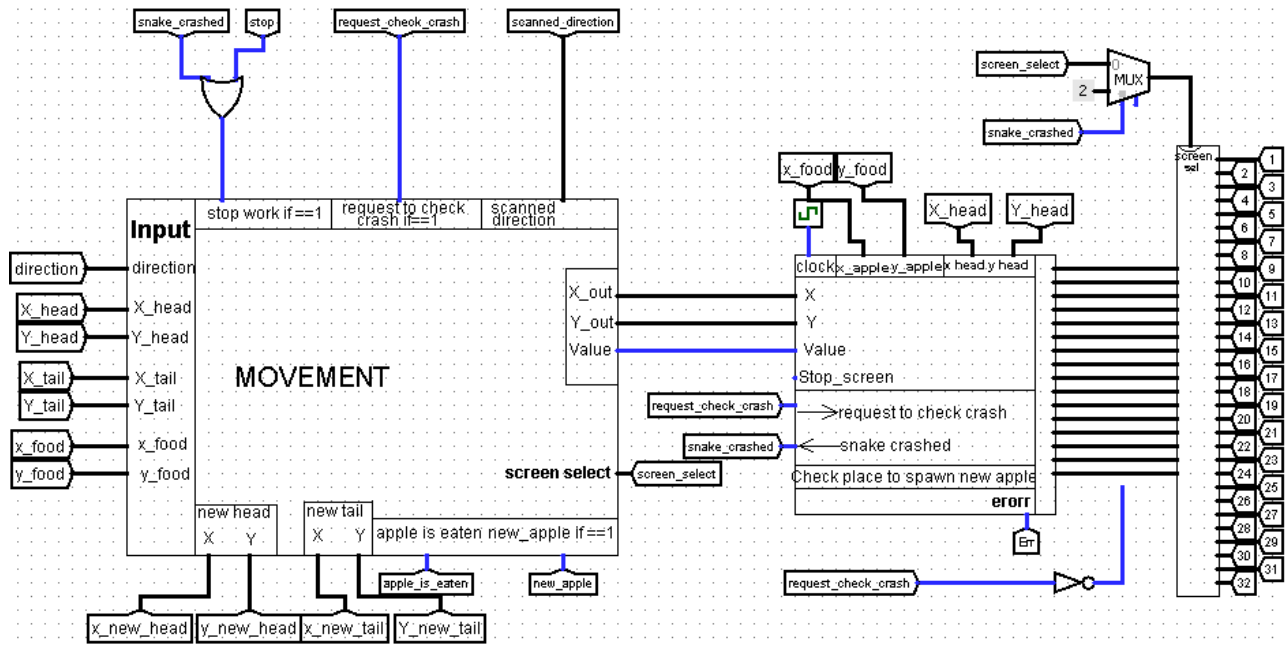


Рисунок 4. Модуль Movement.

Модуль Movement - логика перемещения. Получает и обрабатывает данные и выдает новые координаты головы и хвоста, координаты и значения для записи на экран, а также сигналы. Блок дисплея получает X,Y, значение Value и по тактовому сигналу обновляет нужный пиксель. Через MUX по сигналу screen\_select выбирается какой экран отображать. Сигнал snake\_crashed переключает на экран GAME OVER.

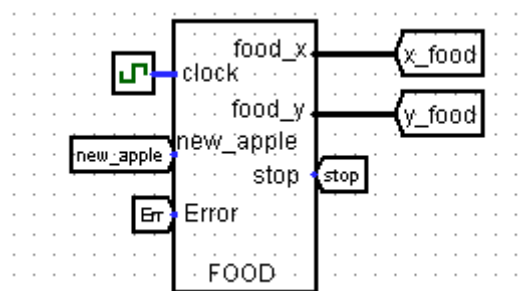


Рисунок 5. Модуль FOOD (GenerateApple)



Модуль FOOD - генерация координат яблока. На вход получает тактовый сигнал (clock), команду на создание нового яблока (new\_apple), сигнал ошибки (Error - яблоко появилось на теле змейки) и сигнал остановки (stop). На выходе – координаты яблока (x\_food, y\_food)

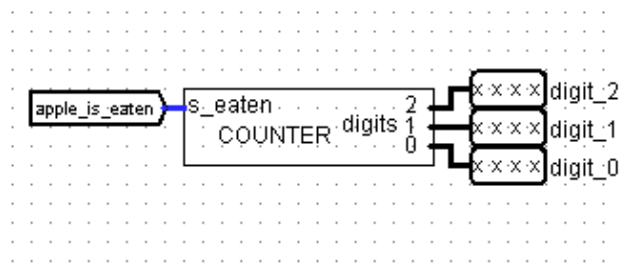


Рисунок 6. Модуль счетчика очков (CounterScore)

Счётчик очков - увеличивается при поедании яблока.

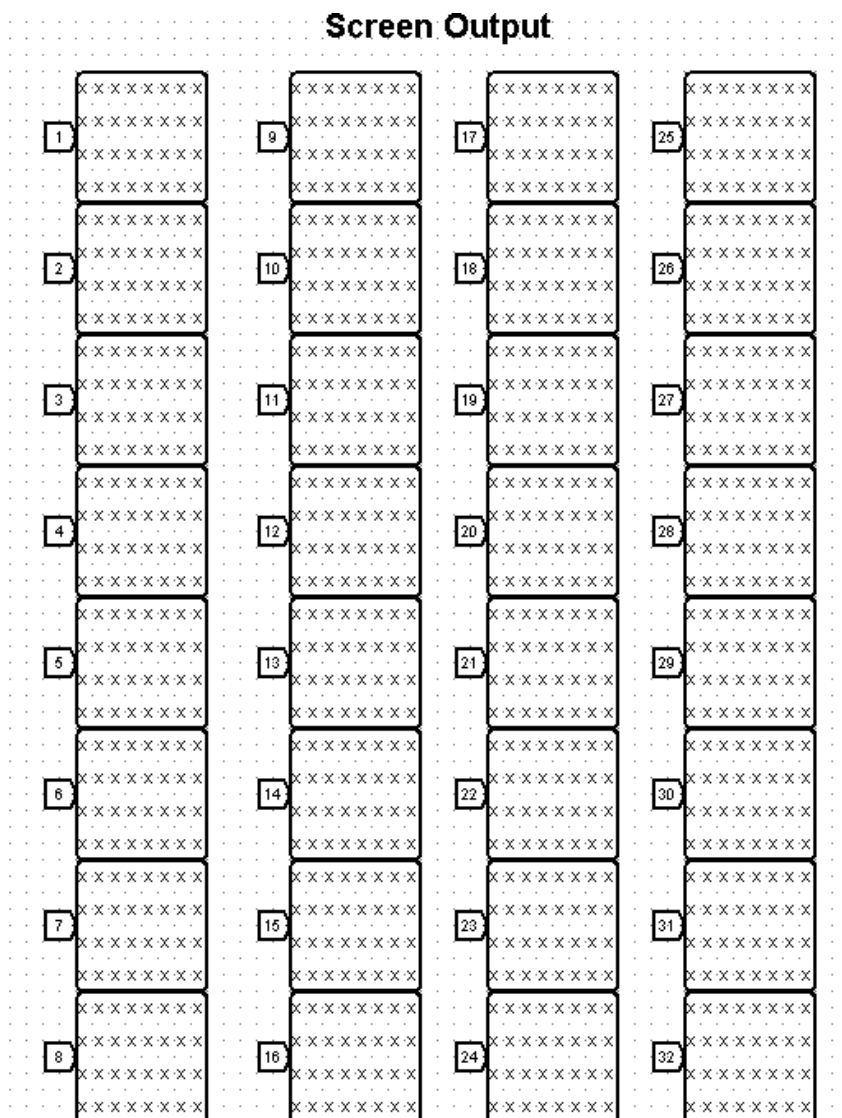


Рисунок 7. Матрица экрана.

Матрица экрана - хранит текущее изображение поля.

Модуль Game связывает эти блоки между собой: направление передается в Movement, результаты Movement обновляют координаты и экран, сигнал о поедании яблока запускает генерацию нового и увеличивает счёт. При столкновении змейки игра переключается на экран GAME OVER.

### 5.3. Модуль движения (Movement)

Модуль Movement управляет всей логикой перемещения змейки и координирует работу процессора CDM-8 с остальными блоками игры.

Что получает на вход (input):

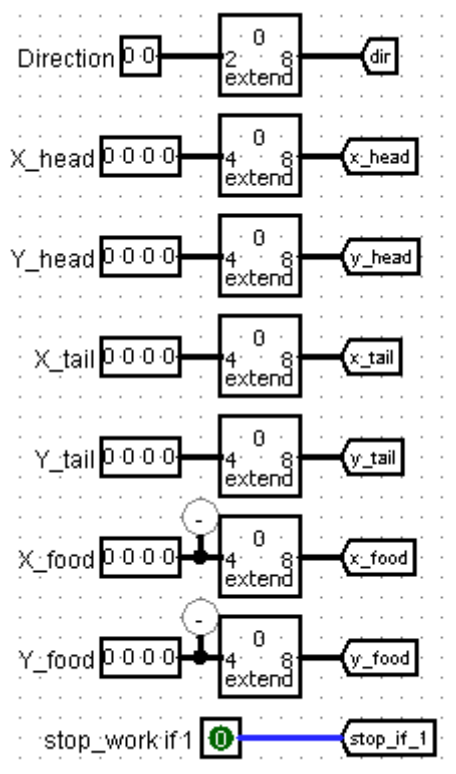


Рисунок 8. input модуля Movement

Direction - текущее направление движения (2бита)

X\_head, Y\_head - координаты головы

X\_tail, Y\_tail - координаты хвоста

X\_food, Y\_food - координаты яблока

stop\_work - сигнал остановки при столкновении.

Все входные значения расширяются до 8 бит для работы с процессором.

Что делает внутри:

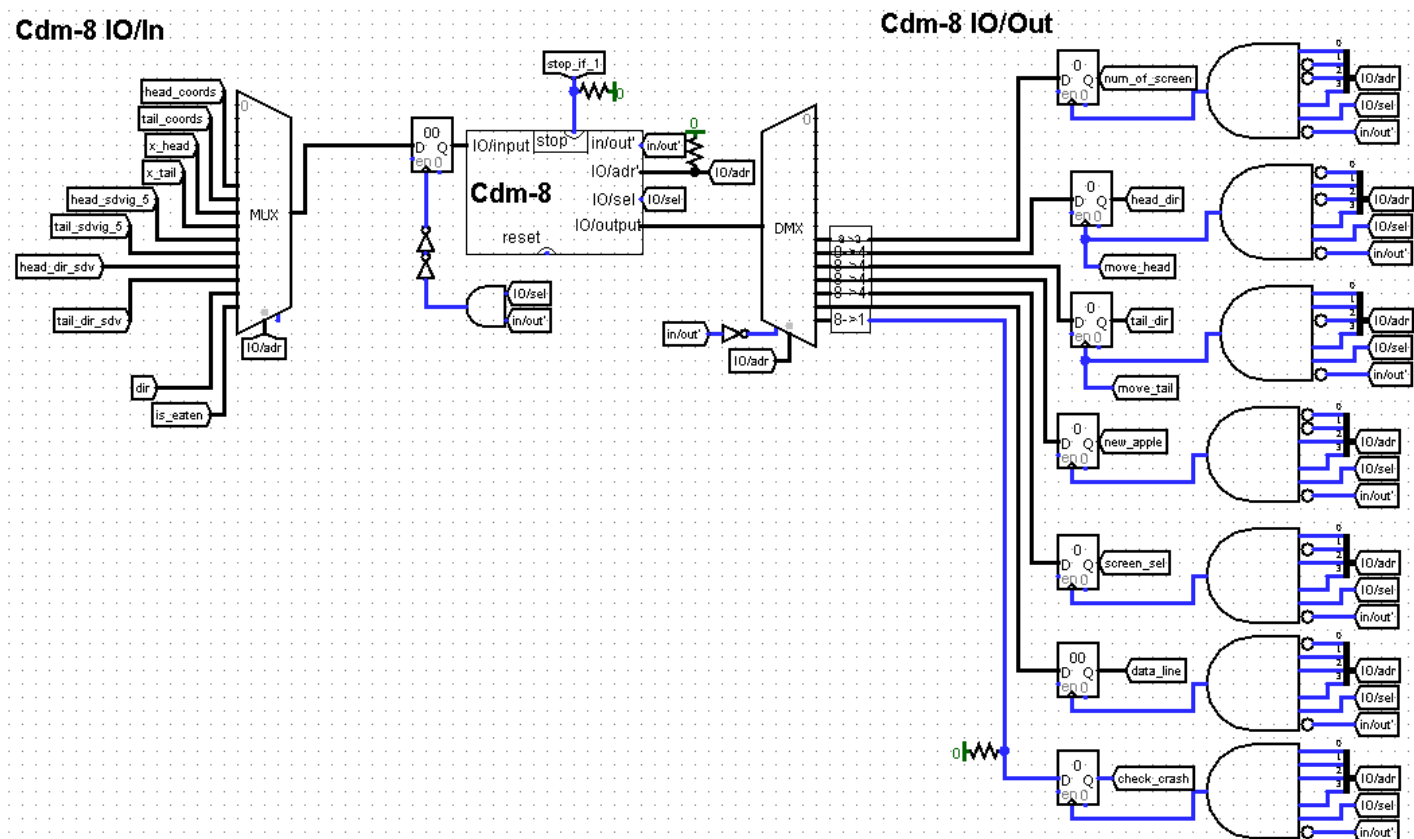


Рисунок 9. Работа с процессором CDM-8

Процессор выполняет программу “snake.asm”. Он получает данные через порт ввода(io/input): координаты головы, хвоста, сдвиги для работы с памятью направлений. Через порт вывода (io/output) процессор отдаёт команды: сдвинуть голову, сдвинуть хвост, создать яблоко, проверить столкновение, переключить экран. Команды распределяются через демультиплексор по соответствующим регистрам.

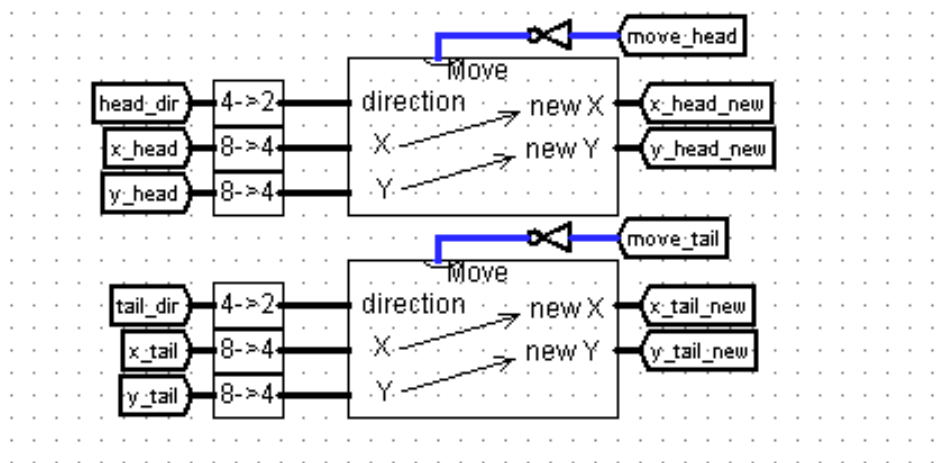


Рисунок 10. Модуль ChangingCoordinates

Изменение координат осуществляется с помощью двух подмодулей move : move\_head и move\_tail, которые принимают текущие координаты и направления, а возвращают две новые вычисленные координаты. Направление приходит как 2 бита, координаты как 4.

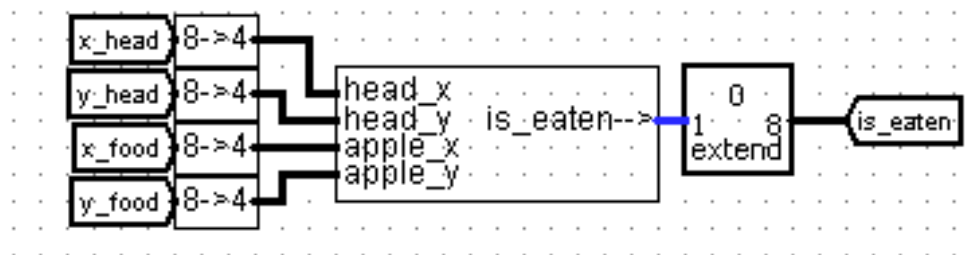


Рисунок 11. Проверка на “съедено ли яблоко”

Проверка “съедено ли яблоко” происходит при помощи сравнения координат головы и яблока, если совпали - выставляется сигнал is\_eaten.

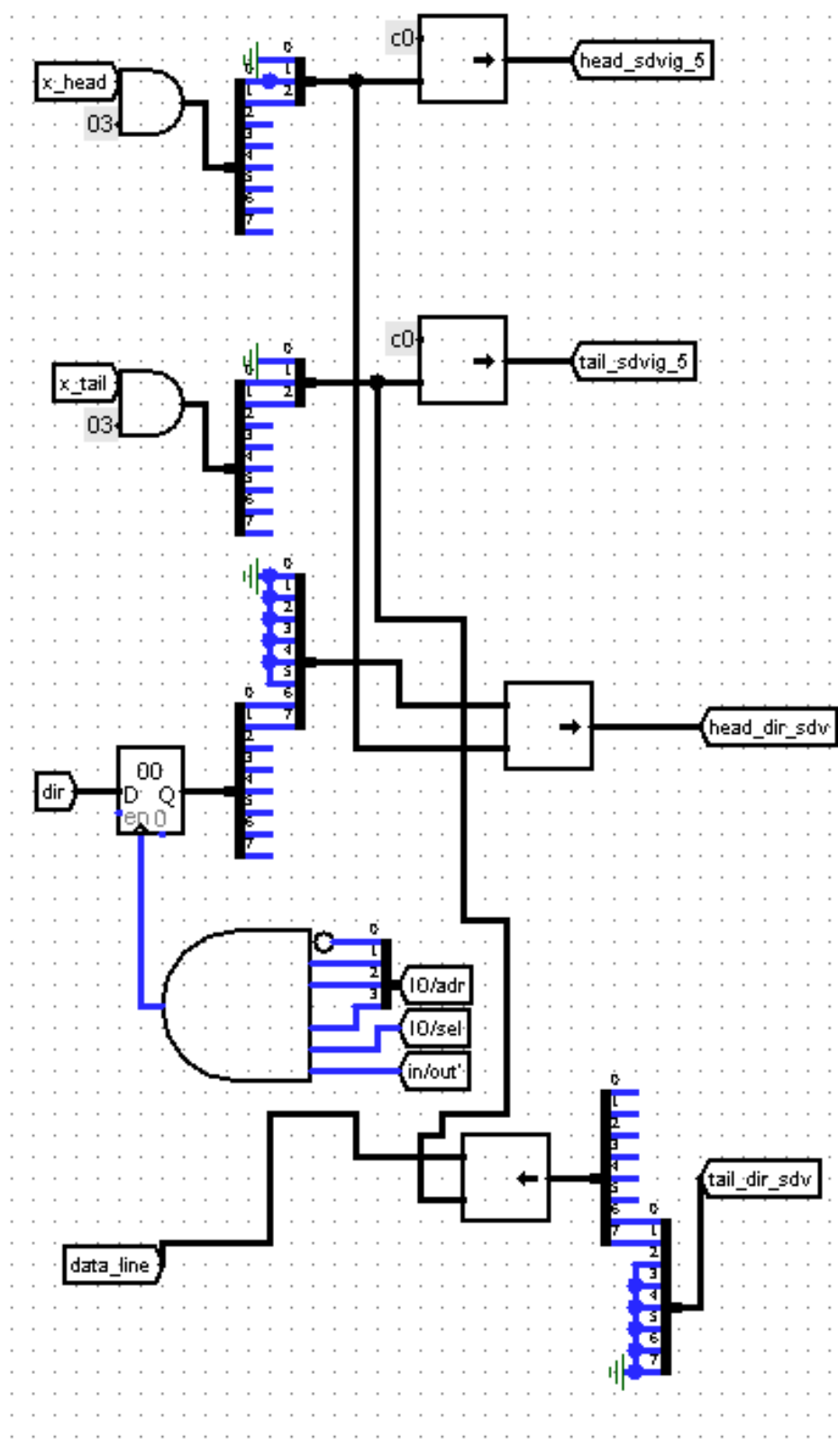


Рисунок 12. Блок “Help CDM-8 take/save direction to RAM”

Для корректного движения хвоста нужно помнить, в каком направлении двигалась голова на каждом шаге. Блок “Help CDM-8 take/save direction to RAM” помогает процессору записывать и считывать направления из памяти.

Что выдает на выход(output):

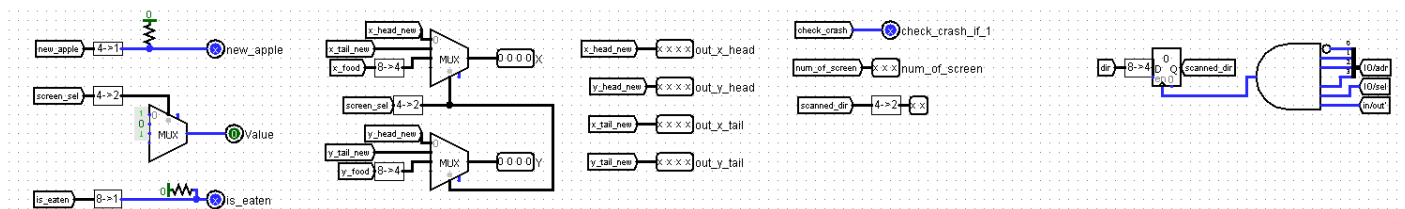


Рисунок 13. output модуля Movement

new\_apple – сигнал генерации нового яблока

is\_eaten – яблоко съедено

value – значение пикселя для записи на экран (через mux выбирается голова, хвост или яблоко)

out\_x\_head, out\_y\_head – новые координаты головы

out\_x\_tail, out\_y\_tail – новые координаты хвоста

check\_crash\_if\_1 - запрос проверки на столкновение

num\_of\_screen - номер текущего экрана(старт/игра/конец)

scanned\_dir - считанное направление для хвоста

## 5.4. Модули управления клавиатурой (Keyboard, EditKeyboardKeys)

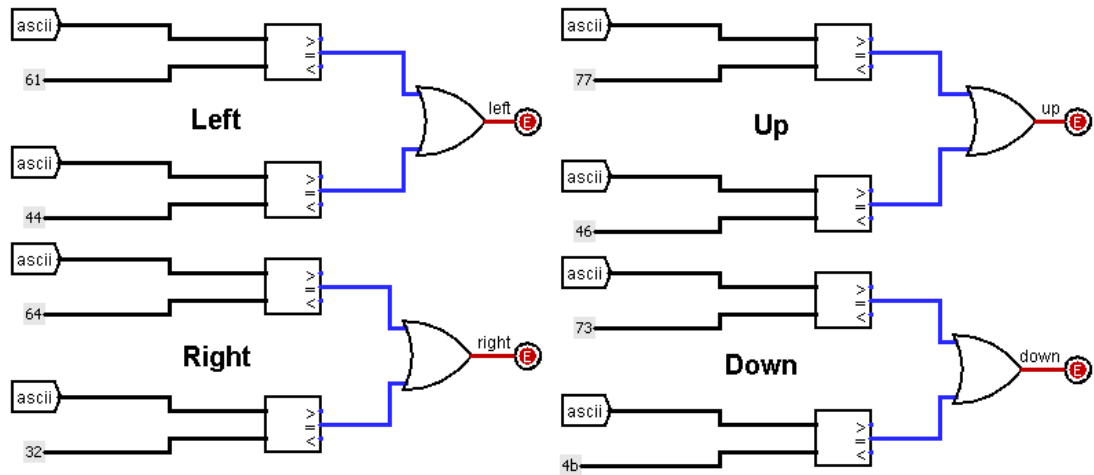


Рисунок 14. Схема ввода данных с клавиатуры (Keyboard)

Схема Keyboard принимает ASCII-код нажатой клавиши и определяет направление. Каждое направление распознаётся по двум клавишам (строчная и заглавная): W/w – вверх, S/s – вниз, A/a – влево, D/d – вправо. На выходе формируются четыре однобитных сигнала : up, down, left, right.

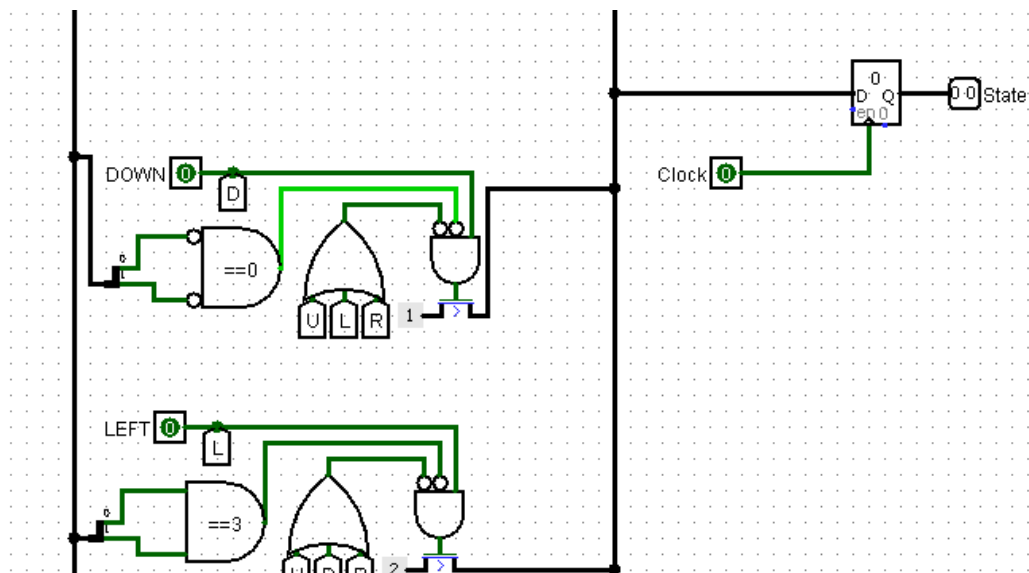


Рисунок 15. Схема фильтрации направления движения (EditKeyboardKeys)

Схема EditKeyboardKeys фильтрует ввод, предотвращая разворот змейки на 180°. Если змейка движется вправо, то нажатие влево игнорируется, если вверх, то нажатие вниз игнорируется. Для этого текущее направление(2бита) сравнивается с противоположным, и при совпадении нажатие не проходит. Итоговое направление сохраняется в регистре State и передается в модуль Movement.

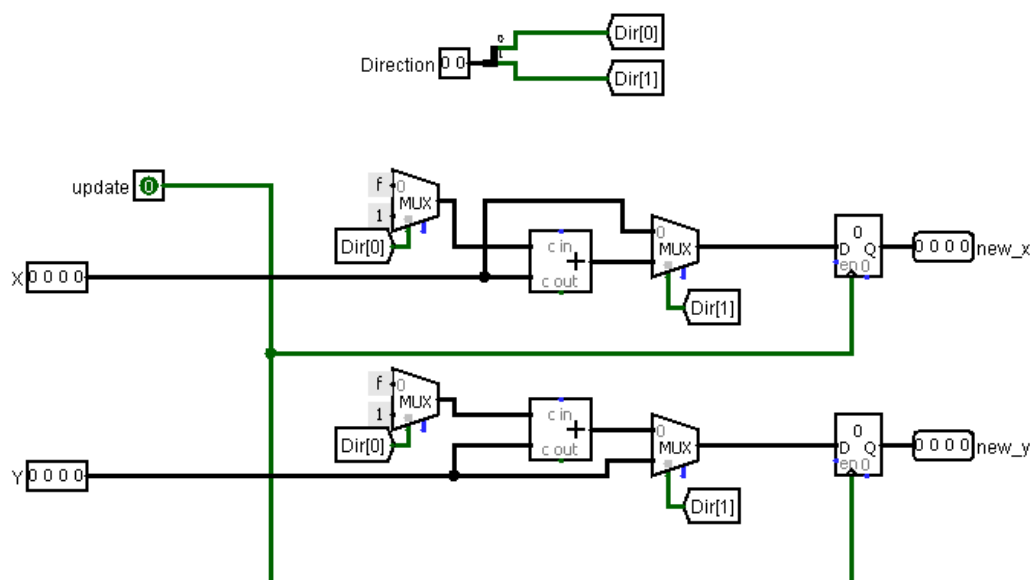


Рисунок 16. Схема изменения координат (ChangingCoordinates)

Эта схема на вход получает координаты X и Y (4бита каждая), направление direction(2бита) и сигнал update. Направление разбивается на два бита: Dir[0] определяет ось (X или Y), Dir[1] - сторону( влево(+), или вправо(-) )

Для каждой координаты через мультиплексор выбирается +1 или -1, затем сумматором прибавляется к текущему значению. Второй мультиплексор по Dir[1] решает, применять ли изменение к данной оси. Результат записывается по сигналу update. На выходе получаем новые координаты: new\_x и new\_y.



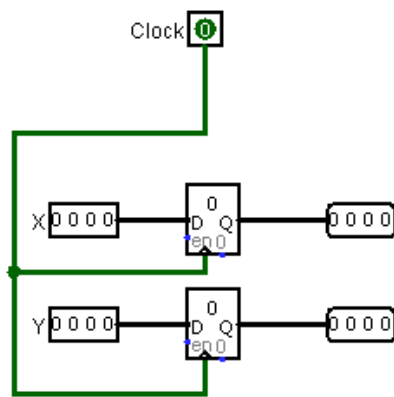


Рисунок 17. Схема сохранения изменений координат (SaveCoordinates)

Схема **SaveCoordinates** выполняет хранение координат сегментов змейки. Благодаря этому сохраняется целостность формы змейки и корректно отображается её перемещение.

## 5.5. Схема генерации яблок (GenerateApple)

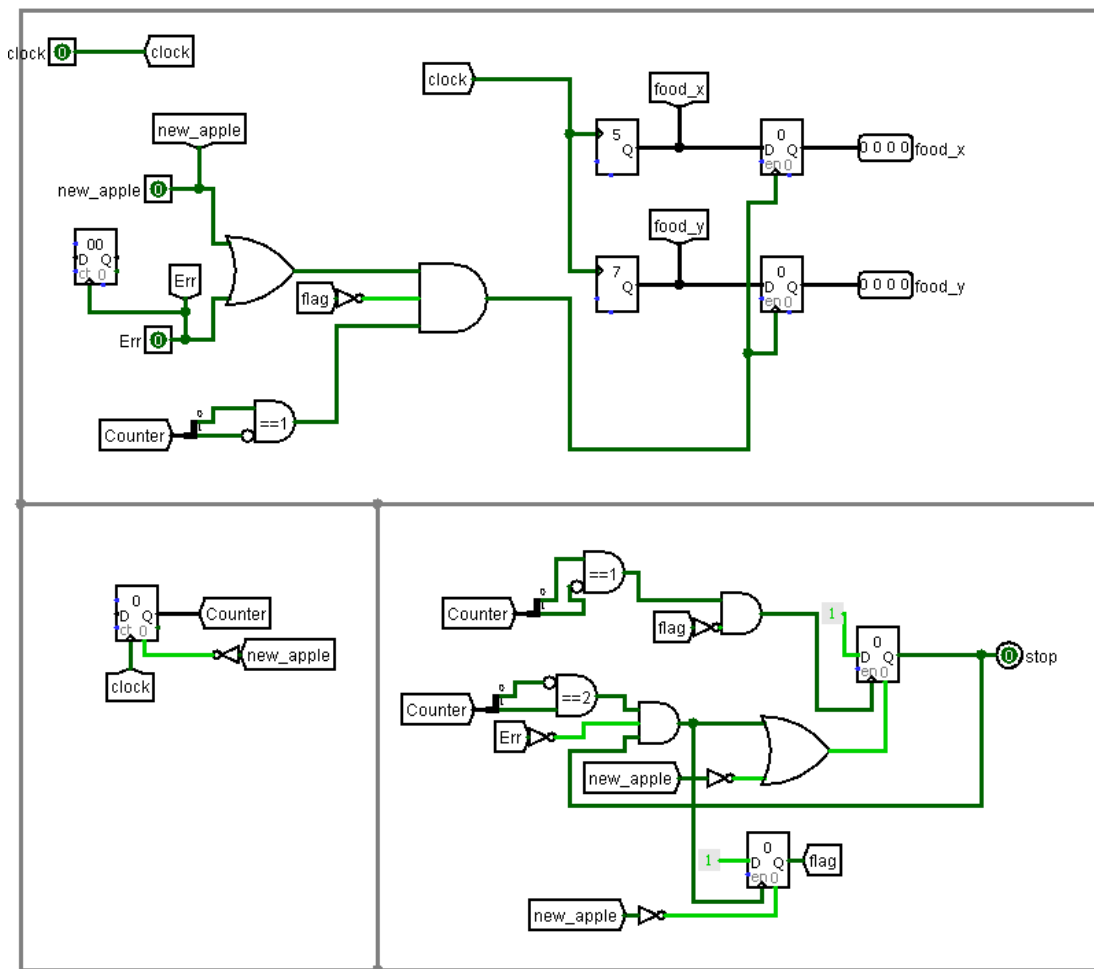


Рисунок 18. Схема генерации яблока (GenerateApple)

Схема GenerateApple отвечает за создание новых координат яблока на игровом поле.

На вход получает: сигнал new\_apple (запрос на генерацию), тактовый сигнал clock и сигнал ошибки Err(яблоко появилось на теле змейки). На выходе получаем координаты food\_x и food\_y.

Координаты генерируются с помощью двух счётчиков (counter), которые непрерывно считают по тактовому сигналу. В момент получения сигнала new\_apple текущие значения счетчиков фиксируются в регистрах как новые координаты яблока. Поскольку счетчики работают на высокой частоте, момент остановки фактически случаен - это создает эффект псевдослучайной генерации.

## 5.6. Схема проверки поедания яблока (CheckEatApple)

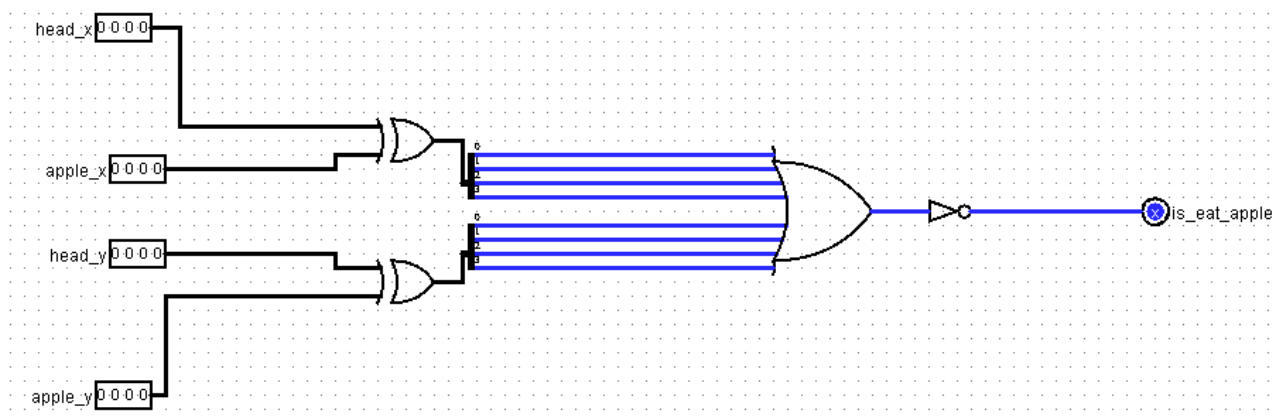


Рисунок 19. Схема проверки поедания яблока (CheckEatApple)

Схема **CheckEatApple** определяет, совпадают ли координаты головы змейки с координатами яблока.

На вход схема получает координаты головы (head\_x, head\_y) и яблока (apple\_x, apple\_y). На выходе получаем сигнал is\_eat\_apple.

Работает эта схема через побитовое сравнение: координаты head\_x и apple\_x, а также head\_y и apple\_y попарно подаются на XOR-gate. Если биты одинаковые – XOR дает 0. Результаты всех XOR объединяются через OR-gate, а затем инвертируется. Если все биты обеих координат совпали - на выходе 1, значит яблоко съедено.

## 5.7. Счетчик очков (CounterScore)

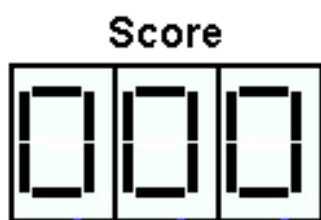


Рисунок 19. Счетчик очков

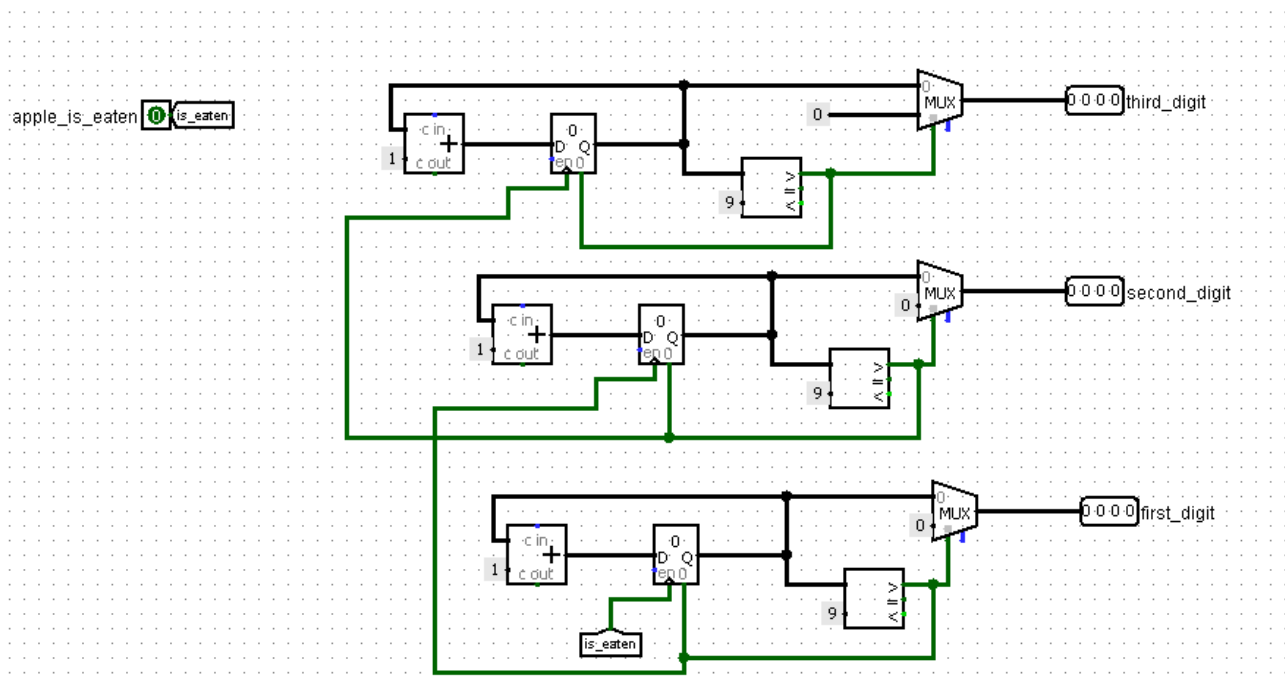


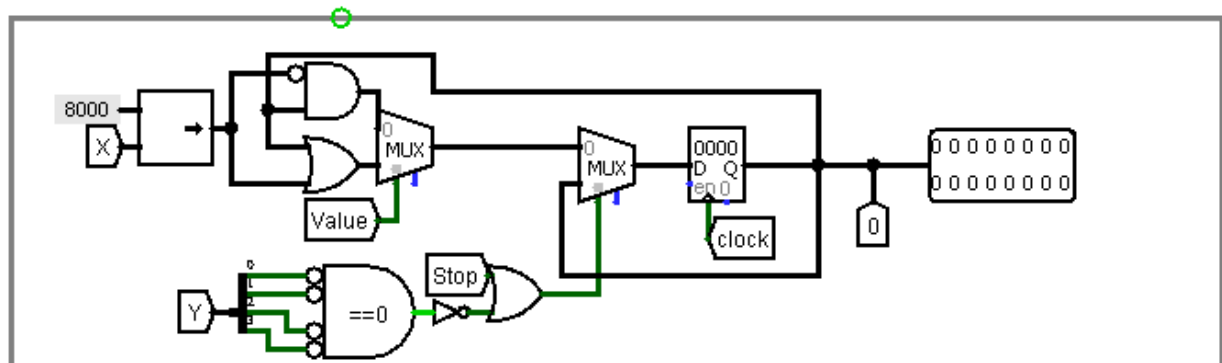
Рисунок 20. Схема счетчика очков (CounterScore)

Схема CounterSpace подсчитывает количество съеденных яблок и выводит результат в виде трёхразрядного десятичного числа.

На вход схема получает сигнал `is_eater` (съедено ли яблоко). На выходе – три значения, где : `first_digit` - единицы, `second_digit` - десятки, `third_digit` - сотни.

Схема построена как каскад из трех одинаковых блоков: каждый блок содержит сумматор(+1), D-регистр для хранения текущего значения и компаратор с числом 9. Когда разряд достигает 9, при следующем инкременте он сбрасывается в 0 через мультиплексор, а сигнал переноса передается на следующий разряд.

## 5.8. Схемы отображения (DataScreen, UpdateScreen)

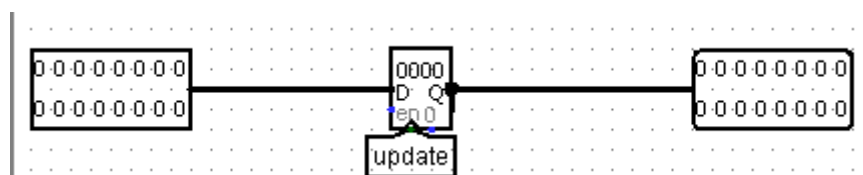


## Рисунок 21. Схема отображения (DataScreen)

Схема DataScreen хранит состояние игрового поля в виде матрицы.

На вход получает координаты X,Y, значение value (записать/стереть пиксель), тактовый сигнал clock и сигнал stop. На выходе – 16 восьмибитных регистров, каждый представляет одну строку экрана.

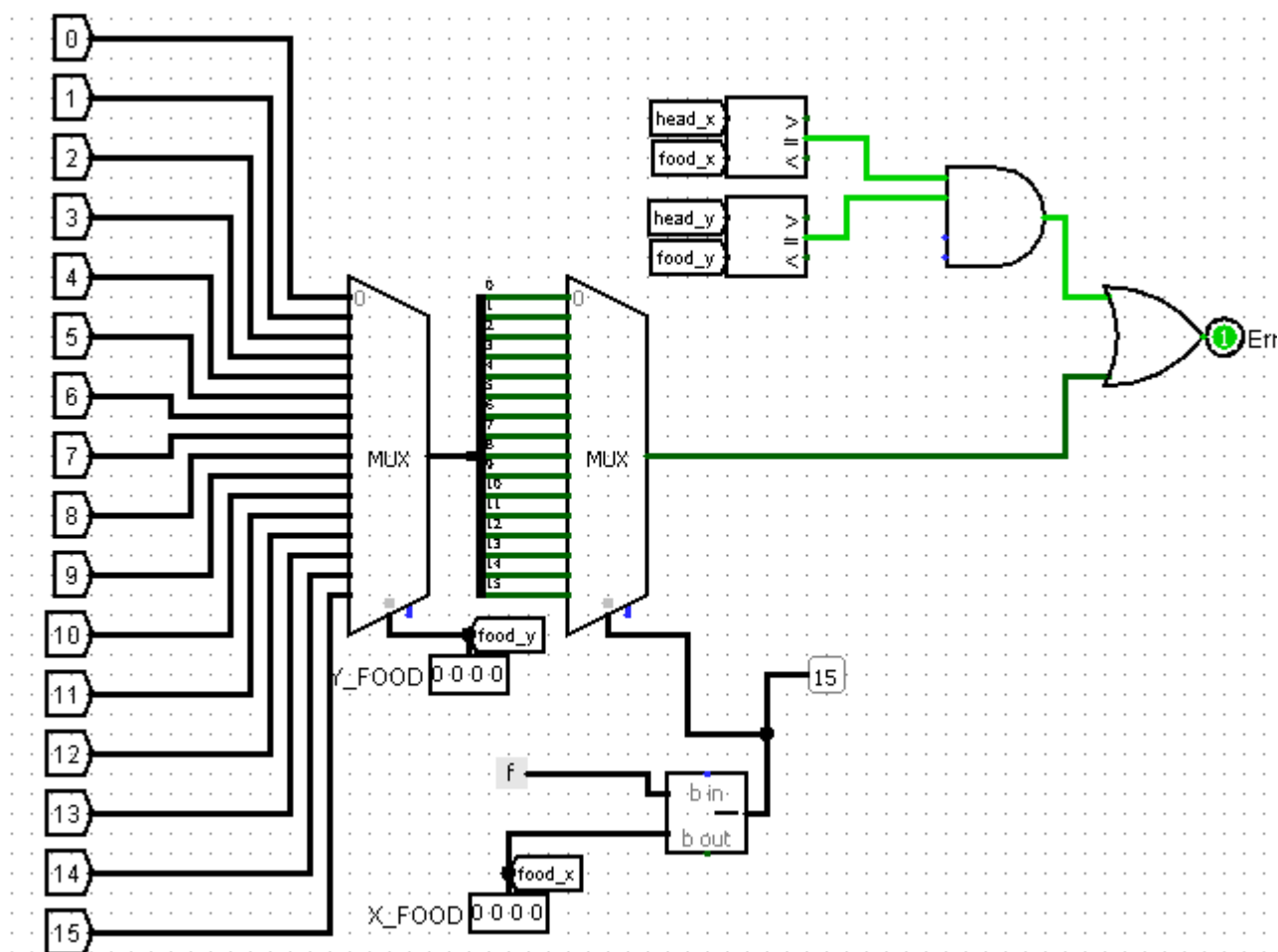
Схема состоит из 16 одинаковых блоков. Каждый блок отвечает за свою строку : сравнивает входной Y со своим номером, и, если совпал, записывает или стирает бит в позиции X. Запись происходит через мультиплексоры: координата X декодируется в позицию бита, value определяет записать 1 или 0, результат сохраняется в 8-битный регистр по сигналу clock. Сигнал stop блокирует обновление при завершении игры.



## Рисунок 22. Схема обновления данных экрана

Схема update\_to\_screen передает данные из матрицы DataScreen на светодиодный дисплей. Состоит из 16 одинаковых блоков, по одному на каждую строку. Каждый блок содержит 8-битный D регистр, который принимает данные строки из DataScreen и по сигналу update фиксирует их для вывода на экран.

DataScreen может обновляться в любой момент, но на экран данные попадают только по сигналу update. Это обеспечивает синхронное обновление всего изображения за один такт.



По координатам яблока (X\_FOOD, Y\_FOOD) проверяется, занята ли клетка. Дополнительно сравниваются координаты яблока и головы. Если клетка занята, то  $egg = 1$  – выход, яблоко регенерируется.

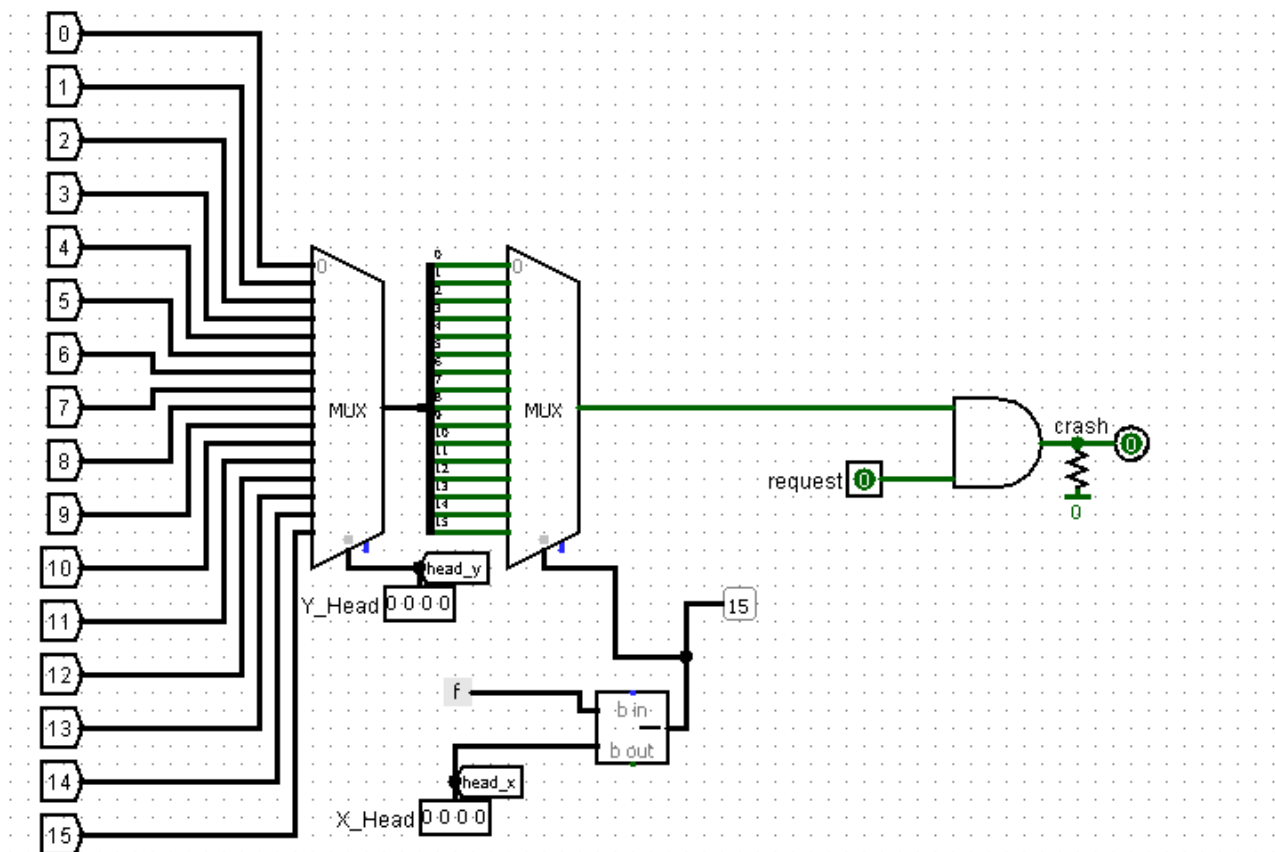


Рисунок 24. Схема проверки столкновения головы змеи с телом.

Логика та же самая. По координатам головы (X\_head, Y\_Head) через мультиплексоры выбирается нужная строка и бит, если бит = 1, то есть клетка занята телом и пришел сигнал request, то crash = 1, что означает завершение игры.

Request – сигнал от процессора, который говорит “проверь столкновение сейчас”, некое разрешение на проверку в нужный момент. Процессор его отправляет после того, как голова переместилась на новую клетку. Без него проверка не срабатывает – crash был бы постоянно 1, т.к голова как бы всегда “на теле”, ведь на она сама часть змейки.

## 5.10. Модули игровых состояний

Схема ChooseScreen переключает отображение между тремя экранами в зависимости от сигнала screen\_select :

0 – экран начала, используется до начала игры.(START\_SCREEN)

1 – основной игровой экран.

2 – экран окончания, используется после возникновения проигрышной ситуации (GAME\_OVER)

Схема состоит из 32 мультиплексоров (по одному на каждую строку дисплея). Каждый мультиплексор принимает три варианта данных строки (start, game, game\_over) и по значению screen\_select выбирает, какой из них передать на выход (line\_1, ..., line\_32). Так обеспечивается мгновенное переключение экранов без перерисовки.

Главный игровой экран – принимает 16 строк из DataScreen и расширяет их с 16 до 32 бит для вывода на дисплей.

START\_SCREEN – содержит заранее прошитое изображение стартового экрана. Данные хранятся как константы в 32 регистрах.

GAME\_OVER – аналогично START\_SCREEN.

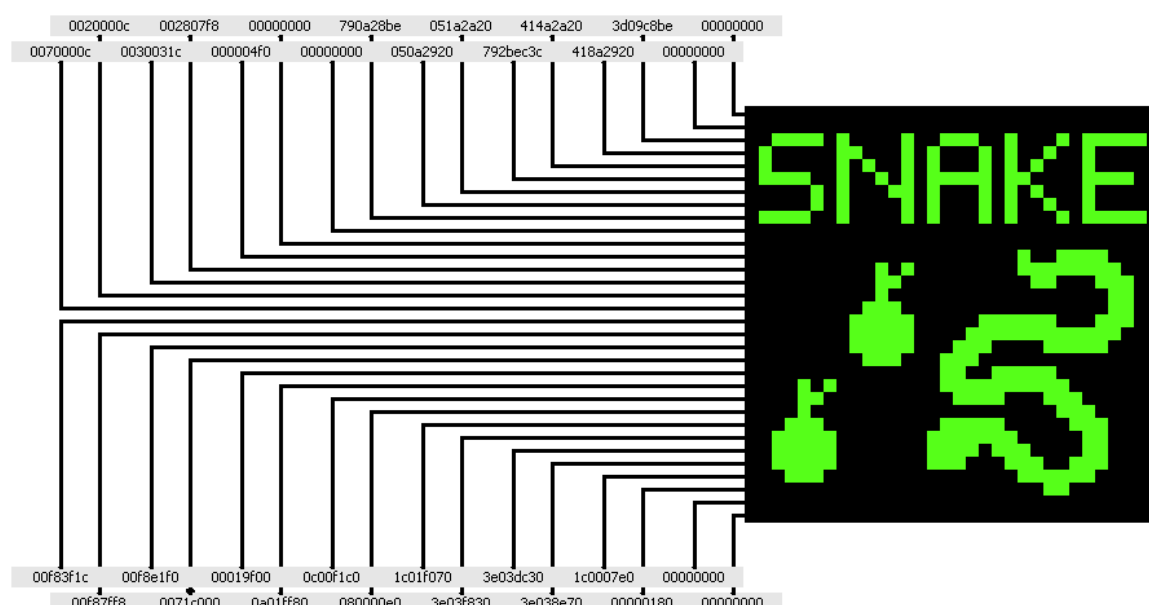


Рисунок 25. Экран начала игры



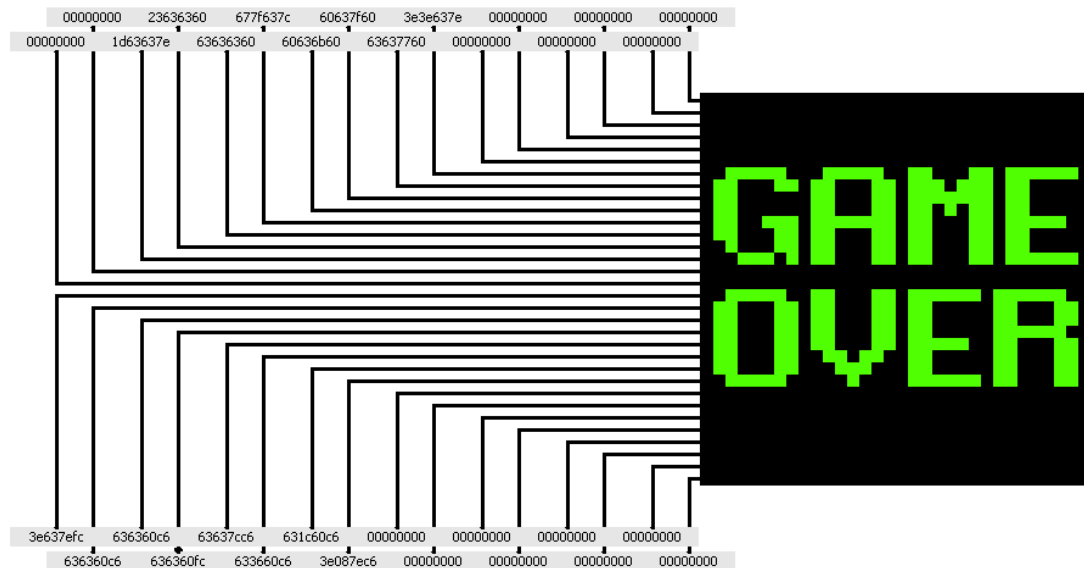


Рисунок 26. Экран окончания игры

## 5.11. Процессорный модуль (CDM-8)

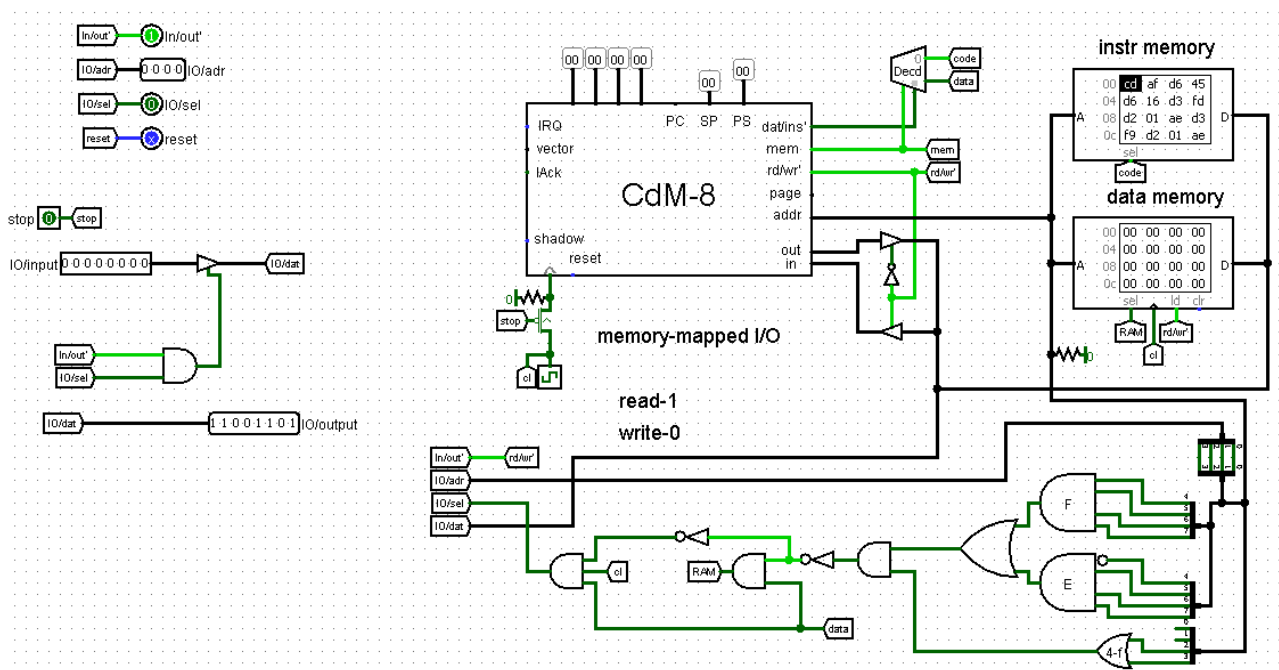


Рисунок 27. Схема подключения процессора cdm-8

Процессор CDM-8 (mark4) – 8битный процессор, который выполняет программу snake.asm, которая управляет последовательностью игровых действий.

Процессор подключен к двум блокам памяти:

instr memory(память инструкций) – содержит скомпилированный код игры (snake.img).

Процессор читает из неё команды для выполнения.

Data memory(память данных) – используется для хранения направлений движения змейки на каждом шаге (для корректного движения хвоста)

Взаимодействия с игровыми модулями происходит через memory-mapped I/O (ввод-вывод через адресное пространство). Процессор обменивается данными с периферией через следующие сигналы:

io/input – процессор читает данные от периферии (координаты, направления, результаты проверок)

io/output – процессор отправляет команды модулям (сдвинуть голову, сдвинуть хвост, создать яблоко, проверить столкновение)

io/adr – адрес порта (определяет, к какому модулю идет обращение)

io/sel – сигнал выбора устройства

io/out' – направление обмена (read = 1, write = 0)

Сигнал stop останавливает процесс при завершении игры. Сигнал reset перезапускает процессор. Адресное пространство портов (0xE6–0xFF) распределено между модулями через демультиплексоры в схеме Movement, где каждый адрес соответствует конкретной команде или запросу данных.

В основном цикле программы snake.asm поочередно вызываются подпрограммы перемещения головы и хвоста. В коде реализованы следующие подпрограммы:

move\_head – перемещение головы змейки и проверка факта поедания яблока

move\_tail – обновление положения хвоста

spawn\_apple – генерация нового яблока

load\_direction\_to\_mem – запись направления движения в память

take\_direction\_from\_mem - извлечение направления из памяти

Для взаимодействия с Logisim используются специальные адреса памяти ввода-вывода, позволяющие считывать направление движения, обновлять координаты, запускать генерацию яблок и переключать игровые экраны.

## 6. Программная часть

### 6.1. Структура программы “snake.asm”

Программа snake.asm написана на языке ассемблера CDM-8 и выполняется процессором внутри схемы Movement. Программа управляет последовательностью игровых действий : определяет порядок перемещения змейки, проверки столкновений и генерации яблок.

Программа состоит из блока инициализации, основного цикла и пяти подпрограмм.

Инициализация (start):

При запуске устанавливается указатель стека, создается первое яблоко, выполняется начальное перемещение головы и экран переключается на игровой режим.

Основной цикл (main):

Бесконечный цикл, в котором каждый такт последовательно вызываются подпрограммы move\_head и move\_tail. Цикл выполняется до столкновения змейки.

Подпрограммы :

move\_head - считывает текущее направление, сохраняет его в память, перемещает координаты головы, проверяет поедание яблока. Если яблоко съедено - создает новое, если нет - проверяет столкновение и отрисовывает голову на экране

Move\_tail - считывает сохраненное направление из памяти, перемещает координаты хвоста и стирает старый пиксель хвоста с экрана

Spawn\_apple - отправляет команду на генерацию новых координат яблока и отрисовывает его на экране

Load\_direction\_to\_mem - сохраняет текущее направление головы в ячейку памяти данных. Использует маски для очистки старого значения и записи нового

Take\_direction\_from\_mem - извлекает ранее сохраненное направление из памяти для корректного движения хвоста

## 6.2. Взаимодействие программы со схемами

Таблица портов показывает через какие адреса программа общается со схемами.

| Адрес | Тип    | Назначение                                                 |
|-------|--------|------------------------------------------------------------|
| 0xE6  | Чтение | Получить адрес ячейки памяти для записи направления головы |
| 0xE7  | Чтение | Получить адрес ячейки памяти для чтения направления хвоста |
| 0xEA  | Чтение | Маска очистки направления в ячейке                         |
| 0xEB  | Чтение | Маска извлечения направления из ячейки                     |
| 0xEC  | Чтение | Направление, сдвинутое в нужную позицию для записи         |
| 0xED  | Чтение | Извлечённое направление для хвоста                         |
| 0xEE  | Чтение | Текущее направление движения (от клавиатуры)               |
| 0xEF  | Чтение | Флаг: яблоко съедено (1 = да)                              |
| 0xF9  | Запись | Переключение на игровой экран                              |
| 0xFA  | Запись | Команда: переместить координаты головы                     |
| 0xFB  | Запись | Команда: переместить координаты хвоста                     |
| 0xFC  | Запись | Команда: сгенерировать новое яблоко                        |
| 0xFD  | Запись | Выбор регистра отрисовки (0=голова, 1=хвост, 2=яблоко)     |
| 0xFE  | Запись | Загрузка строки данных направления                         |
| 0xFF  | Запись | Запрос проверки столкновения                               |

```
5  start:
6      setsp 0xaf
7      jsr spawn_apple      # создание яблока в начале игры
8      jsr move_head        # перемещение головы
9      ldi r3, 0xfd         # установка регистра выбора в 1
10     ldi r2, 1
11     st r3, r2
12     ldi r3, 0xf9         # переключение экрана на основной игровой
13     ldi r2, 1
14     st r3, r2
15
16  main:
17     jsr move_head        # работа с головой
18     jsr move_tail        # работа с хвостом
19     br main              # возврат в начало основного цикла
20
```

Рисунок 28. Блок кода 1

start - инициализация игры:

вызывает spawn\_apple -> схема GenerateApple создает первое яблоко.

вызывает move\_head -> схема ChangingCoordinates вычисляет начальную позицию.

записывает в порт 0xf9 -> схема ChooseScreen переключает на игровой экран.

main - основной цикл :

вызывает move\_head и move\_tail каждый такт.

цикл повторяется бесконечно до столкновения.

```

21  # подпрограммы
22      move_head: # подпрограмма перемещения головы
23          # считывание направления
24          ldi r2, 0xee
25          ld r2, r2
26          # сохранение направления в память
27          jsr load_direction_to_mem
28          # перемещение координат головы
29          ldi r3, 0xfa
30          st r3, r2
31          # проверка, съедено ли яблоко
32          if
33              # запрос к Logisim
34              ldi r3, 0xef
35              ld r3, r3
36              tst r3
37          is nz # если яблоко съедено
38              jsr spawn_apple
39              br move_head # возврат к началу подпрограммы
40          else
41              # проверка столкновения
42              ldi r3, 0xff
43              ldi r2, 1
44              st r3, r2
45              ldi r3, 0xff
46              ldi r2, 0
47              st r3, r2
48          fi
49          ldi r3, 0xfd # отрисовка пикселя с новыми координатами
50          ldi r2, 0     # установка регистра выбора в 0
51          st r3, r2
52          rts
53

```

Рисунок 29. Блок кода 2

move\_head - перемещение головы:

читает порт 0xEE -> схема EditKeyboardKeys выдает текущее направление

вызывает load\_direction\_to\_mem -> сохраняет направление в Data memory

записывает в порт 0xFA -> схема ChangingCoordinates вычисляет новые координаты головы

читает порт 0xEF -> схема CheckEatApple сообщает, съедено ли яблоко

если съедено -> вызывает spawn\_apple

если не съедено -> записывает в порт 0xFF -> схема DataScreen проверяет

столкновение записывает в порт 0xFD -> схема DataScreen отрисовывает пиксель головы

```

54      move_tail:
55          jsr take_direction_from_mem    # считывание направления
56          ldi r2, 0xfb                  # перемещение координат хвоста
57          st r2, r3
58          ldi r3, 0xfd                  # стирание пикселя с текущими координатами хвоста
59          ldi r2, 1                      # установка регистра выбора в 1
60          st r3, r2
61          rts
62
63      spawn_apple: # подпрограмма создания яблока
64          # запрос к Logisim для генерации новых координат
65          ldi r3, 0xfc
66          ldi r2, 1
67          st r3, r2
68          ldi r3, 0xfc
69          ldi r2, 0
70          st r3, r2
71          ldi r3, 0xfd                  # отрисовка пикселя-яблока
72          ldi r2, 2                      # установка регистра выбора в 2
73          st r3, r2
74          rts
75

```

### Рисунок 30. Блок кода 3

move\_tail - перемещение хвоста:

вызывает take\_direction\_from\_mem -> читает сохранённое направление из Data memory

записывает в порт 0xFB -> схема ChangingCoordinates вычисляет новые координаты хвоста

записывает в порт 0xFD -> схема DataScreen стирает пиксель хвоста

spawn\_apple - генерация яблока:

записывает в порт 0xFC -> схема GenerateApple фиксирует новые координаты

записывает в порт 0xFD -> схема DataScreen отрисовывает пиксель яблока

```

76     load_direction_to_mem:
77         # r2 - направление
78         # сохранение в стек
79         push r2
80         # запрос адреса в памяти у Logisim
81         ldi r1, 0xe6
82         ld r1, r1
83         # сохранение координат в строке памяти
84         # очистка направления в ячейке
85         ldi r3, 0xea
86         ld r3, r3
87         # инверсия битов
88         not r3
89         # маскирование для удаления старого направления
90         ld r1, r0
91         and r3, r0
92         # перемещение направления в нужное место
93         ldi r2, 0xec
94         ld r2, r2
95         # объединение в строке данных
96         or r2, r0
97         # сохранение в ОЗУ
98         st r1, r0
99         # восстановление из стека
100        pop r2
101        rts
102
103    take_direction_from_mem:
104        # запрос адреса в памяти у Logisim
105        ldi r1, 0xe7
106        ld r1, r1
107        # извлечение направления из строки данных
108        ldi r3, 0xeb
109        ld r3, r3
110        # маскирование лишней информации в строке (через AND)
111        ld r1, r0
112        and r0, r3
113        # загрузка строки данных в Logisim
114        ldi r2, 0xfe
115        st r2, r3
116        # загрузка направления для хвоста
117        ldi r3, 0xed
118        ld r3, r3
119        rts
120    end

```

Рисунок 31. Блок кода 4



load\_direction\_to\_mem - сохранение направления:

читает порт 0xE6 -> схема SaveCoordinates (в Movement) выдаёт адрес ячейки

читает порт 0xEA -> получает маску очистки

читает порт 0xEC -> получает сдвинутое направление

записывает результат в Data memory

take\_direction\_from\_mem - извлечение направления:

читает порт 0xE7 -> получает адрес ячейки для хвоста

читает порт 0xEB -> получает маску извлечения

записывает в порт 0xFE -> передаёт данные обратно в схему Movement

читает порт 0xED -> получает готовое направление для хвоста

## 7. Инструкция по запуску

Открыть Logisim.

Загрузить файл `snakemain.circ` (Файл -> Открыть)

Перейти в схему CDM8 (двойной клик в дереве проекта)

Загрузить образ программы в память инструкции :

Правой кнопкой по блоку `instr memory` -> загрузить образ -> выбрать файл `snake.img`

Перейти в схему `main`, выбрать инструмент “Нажать”. Нажать на “Нажать на кнопку”, чтоб появился синий овал.

Запустить симуляцию (Ctrl + K)

Управление : W - Вверх. A - Влево. S - Вниз. D - вправо.

Примечание: перед началом игры убедитесь, что тактовая частота установлена достаточно низкой для комфортной игры (Моделирование -> Тактовая частота -> 1 кГц или меньше/больше)

## 8. Заключение

В результате выполнения курсового проекта была разработана аппаратно-программная реализация игры Snake в среде Logisim с использованием процессора CDM-8 и языка Assembler. Реализованы управление движением, хранение координат, генерация яблок, система подсчёта очков, стартовый экран и экран завершения игры. Была обеспечена интеграция программной и аппаратной логики через механизм I/O-взаимодействия. Разработанная архитектура показала возможность создания игровых приложений средствами цифровой схемотехники.

## 9. Источники, использованные при разработке

1. Документация Logisim — <http://www.cburch.com/logisim/docs.html>
2. Assembler CMD-8 — <https://github.com/leadpogrommer/Cdm8-asm>